TYPE 1: RESEARCH PROJECT · TEAM NUMBER 58

An AI-Driven, Blockchain-Anchored Cyber Deception Framework

for Adaptive Honeypot Defense

S.No	Register Number	Student Name
1	CB.SC.U4CSE23111	B Guru Sai Prasad
2	CB.SC.U4CSE23127	N Surya Tejaswar
3	CB.SC.U4CSE23129	N Sisir Reddy
4	CB.SC.U4CSE23611	C Pranav Sharma
5	CB.SC.U4CSE23644	R Vishnu Sathwick

Guide: Dr. Radhika G

Roadmap of the Presentation

Part A — The Problem & The Design

1. Problem statement and motivation
2. Objectives and research gaps
3. Comparison against existing honeypots
4. System architecture (one picture)
5. Life of a single attacker request
6. The five decoy services
7. The three-tier AI response engine
8. Blockchain-anchored logging
9. Implementation stack

Part B — What We Measured

10. Evaluation methodology
11. Module 1: deception engine (4 runs)
12. Module 2: tamper-evident audit trail
13. Module 3: AI verdict accountability
14. Module 4: federated threat intelligence
15. Honest failures we found
16. Project status — 70% complete
17. Discussion, limitations, future work

TAKEAWAY We built *one* deception platform, then evaluated it as *four* separate research modules. Part B is the evidence that the design in Part A actually works — and where it does not.

Problem Statement

Traditional honeypots rely on **static, deterministic response tables** — efficient at scale, but fragile against fingerprinting.

- **Fingerprintable by design.** Prior work shows automated probing can distinguish static honeypots (Cowrie, Kippo, Dionaea) from real systems with near-perfect accuracy in a *single* packet exchange.
- **Single-protocol LLM deception.** Existing LLM-backed honeypots (shelLM, HoneyGPT) are concentrated on SSH/terminal sessions only — no shared framework spans multiple heterogeneous protocols.
- **Untrustworthy forensic logs.** A compromised honeypot host can have its own logs silently edited by the attacker, destroying the evidentiary value of captured telemetry.
- **No accountability for AI decisions.** As LLMs move into the triage loop, a wrong verdict can be quietly edited afterwards — an error laundered into something authoritative.

TAKEAWAY A honeypot must be *believable*, *broad*, and its evidence must be *provable* — current systems deliver at most one of the three.

Objectives

To build a deployable, multi-protocol deception framework that stays convincing under sustained probing and produces a **tamper-evident audit trail**.

- **Cover five distinct attack surfaces** — SSH, an internal AI API, PostgreSQL, an Ethereum JSON-RPC node, and Jenkins CI/CD — behind *one* shared response engine.
- **Stay fast where it can, smart where it must.** Answer routine traffic (≈80%) at millisecond latency via a static path; reserve LLM generation for genuinely novel input.
- **Make every interaction context-aware.** Replace canned responses with an LLM engine that is session-history-aware and intent-classified.
- **Anchor forensic integrity independently of the host.** Hash and commit every log entry to a separate blockchain, so tampering is detectable even if the honeypot itself is compromised.
- **Hold the AI itself accountable.** Anchor the classifier's *verdicts* too, and measure how often they are wrong.
- **Evaluate honestly.** Latency, throughput, diversity and realism as *separate* axes across successively larger datasets — reporting failures, not just favourable numbers.

Research Gaps We Address

- **LLM deception is single-protocol.** shelLM, HoneyGPT and related 2023–2025 work apply LLM response generation only to SSH/terminal sessions; no prior system applies *one* LLM engine uniformly across five heterogeneous protocol types.
- **Tamper-evident logging is generic, not honeypot-specific.** Blockchain hash-anchoring is established in digital forensics (evidence tracking, cloud VM investigation), but we found no honeypot-specific paper combining hash-anchoring with decoy logging.
- **Realism is rarely benchmarked rigorously.** Prior LLM-honeypot papers report a single self-evaluated accuracy figure (e.g. HoneyGPT's self-reported ≈96%); few report latency/throughput trade-offs or scale-dependent failure modes.
- **AI triage output is treated as ground truth.** Work that adds an LLM classifier to security telemetry rarely asks what happens when the model is wrong — or makes the wrong verdict independently verifiable after the fact.
- **Cross-honeypot correlation assumes a central hub.** MISP / STIX-TAXII designs make the aggregator the single machine an adversary must reach to blind the network.

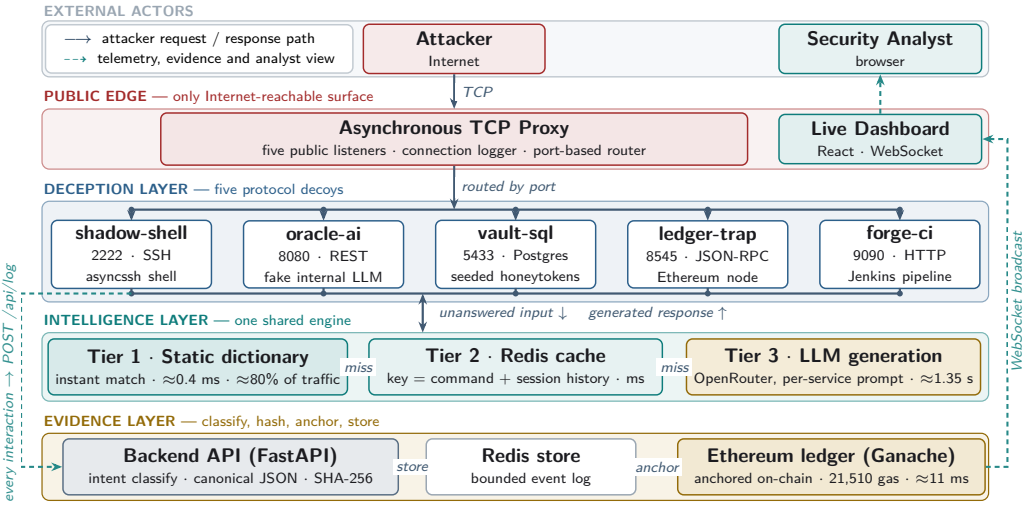
Comparison Against Cited Systems

System	Interaction	AI / LLM Responses	Protocols	Tamper-evident
Cowrie	Medium/High	No (experimental single-tier LLM fallback)	SSH, Telnet	No
T-Pot	Mixed (orchestrator)	No — aggregation only	Broad (bundled daemons)	No
Dionaea	Low–Medium	No	SMB, MySQL, SIP, FTP, TFTP	No
shelLM	High (SSH only)	Yes — full LLM shell session	SSH only	Not addressed
HoneyGPT	High (terminal)	Yes — GPT-3.5/4 via Cowrie proxy	SSH / terminal only	Not addressed
This work	**High, all 5**	**Yes — 3-tier, session-aware, uniform across 5 protocols**	**SSH, AI API, SQL, Web3 RPC, CI/CD**	**Yes — SHA-256 + Ethereum anchor**

Table I, paper Section II.F

TAKEAWAY We are the only row that is simultaneously multi-protocol, LLM-driven *and* forensically verifiable.

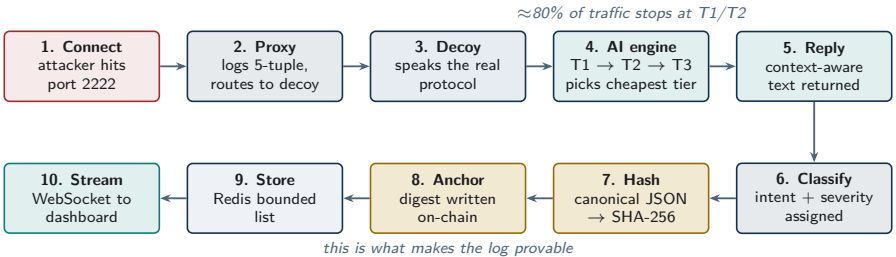
System Architecture



Paper Section III

Life of One Attacker Request

The single slide to remember: what happens between the attacker pressing Enter and the analyst seeing it.



TAKEAWAY Steps 1–5 decide whether the attacker *believes* us. Steps 6–10 decide whether we can *prove* what they did. The two halves are independent, so they are evaluated independently.

The Five Decoy Services

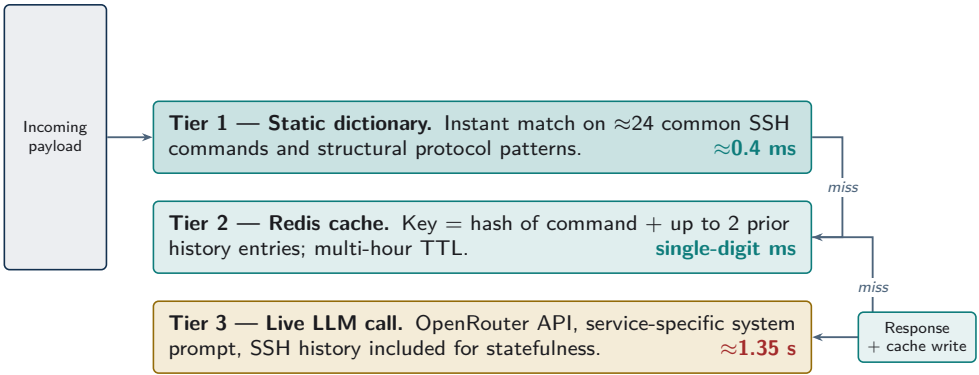
Decoy	Port / Protocol	Role and bait
shadow-shell	2222 / SSH	Real `asyncssh` server; accepts any credentials; shell prompt for `dbadmin@corp-prod-db-01`; every command routes through the AI engine
oracle-ai	8080 / REST	FastAPI `chat/predict` endpoints for a fictitious internal model — prompt-injection bait
vault-sql	5433 / Postgres	Real seeded Postgres instance with synthetic records + honeytoken API keys (OpenAI / GitHub / AWS formats)
ledger-trap	8545 / JSON-RPC	Ethereum node decoy; static replies for common RPC calls, AI engine for uncommon ones
forge-ci	9090 / HTTP	Jenkins-style CI/CD endpoints faking a working pipeline — supply-chain bait

Paper Section III.B

TAKEAWAY Each decoy is a *real* server speaking a *real* protocol — the deception is in the content it returns, not in a fake socket. That is why generic protocol fingerprinting does not immediately expose it.

Three-Tier AI Response Engine

Expensive LLM calls are reserved for genuinely novel input.

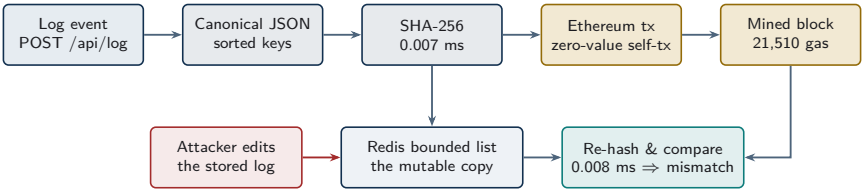


Intent classification: keyword-based, 9 categories — reconnaissance, privilege escalation, SQL injection, reverse shell, cryptomining, crypto theft, prompt injection, supply-chain, unknown. A deliberately simple rule-based baseline; Module 3 tests whether an LLM can beat it.

Paper Section III.C–D

Blockchain-Anchored Logging

Removing the honeypot host from the trust boundary of its own forensic record.



Integrity test, run against the live stack

A log entry (digest `356b7496...33a9a1`) was anchored, fetched back and re-hashed — **identical**. A tampered copy (`"sudo cat /etc/shadow"` → `"ls -la"`) produced digest `7530fb00...0d9d394` — **mismatch**, so the edit is proven.

Paper Section III.E, V.D

Implementation Stack — What Actually Runs

Layer	Technology	Notes
Edge proxy	Python `asyncio` TCP proxy	5 public ports, per-connection logging
Decoys	`asyncssh`, FastAPI, real PostgreSQL	5 containers, real protocol servers
AI engine	Static dict → Redis → OpenRouter	Llama-3.1-8B generator; pooled HTTP
Backend	FastAPI + `web3.py`	classify, hash, anchor, broadcast
Ledger	Ganache v7.9.2, chain ID 1337	Ethereum-compatible, local
Dashboard	React + WebSocket	live attack feed, intent timeline
Orchestration	Docker Compose	one internal network, two exposed services
Evaluation	Custom harnesses + LLM judge (70B)	judge model distinct from generator

TAKEAWAY Everything reported in this deck was measured against *this running stack* — real LLM calls, a real chain, a real database. No simulation, no hand-typed numbers.

Evaluation Methodology

Evaluated against the system's own **static fast-path** (functionally equivalent to Cowrie) vs. its **LLM-backed dynamic path**.

Four metrics, kept separate

- Latency / throughput (30 static, 20 dynamic trials)
- Response diversity — fraction of distinct outputs per service
- LLM-judge realism — independent 70B judge, 1–5 rubric, 30-payload sample
- Blockchain integrity — direct tamper test

Four runs, each scaling realism

1. 100 hand-written payloads (baseline)
2. Same dataset, after an optimisation pass
3. 200 real, publicly-documented payloads
4. 1,328 real payloads (scaled corpora)

TAKEAWAY Explicitly *not* a single composite accuracy figure. Trade-offs and failures are reported, not hidden — which is exactly how the two most interesting results in this deck were found.

Paper Section IV

Module 1, Run 1 — Initial Baseline (100 Hand-Written Payloads)

Metric	Static path	Dynamic (LLM) path
Mean latency	0.4 ms	4,028 ms (4.03 s)
Median latency	0.4 ms	3,529 ms (3.53 s)
Throughput (req/s)	2,500	0.25
Diversity ratio (mean)	≈ 0	0.99 (0.95–1.00)

LLM-judge realism: **4.38 / 5** ($\sigma = 1.28$, $n = 24$ of 30 parsed)

Table II, paper Section V.A

TAKEAWAY The trade-off is stark and it is the whole story of this project: the LLM path is $\approx$10,000$\times$ slower, and $\approx$10,000$\times$ more varied. Everything that follows is about narrowing the first gap without losing the second.

Module 1, Run 2 — Optimisation Pass

Three fixes identified by profiling:

- **HTTP connection pooling** — we were opening a fresh TCP+TLS handshake to OpenRouter on every request.
- **Generation-length cap** tuned 150 → 60 tokens (150 tok ≈5.3 s, 80 tok ≈3.4 s, 40 tok ≈2.0 s — generation length dominates latency).
- **Static SSH dictionary widened** with 12 more reconnaissance commands.

Metric	Before	After	Change
Dynamic mean latency	4.03 s	1.35 s	**≈3× faster**
Dynamic throughput (seq.)	0.25 rps	0.74 rps	**≈3×**
Static throughput	2,500 rps	3,333 rps	**+33%**
LLM-judge realism	4.38/5 (24/30)	4.00/5 (30/30)	**−0.38**

Table III, paper Section V.B

TAKEAWAY Speed was bought with realism — shorter generations are less convincing. We report the loss rather than quoting only the 3× speed-up.

Latency Comparison — Before vs. After

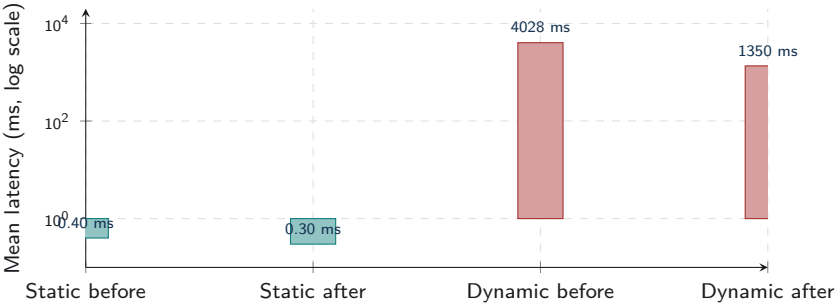


Fig. 1, paper Section V.B

TAKEAWAY Four orders of magnitude separate the two paths. This is why the tiering matters: routing 80% of traffic to the left-hand bars is what makes the system usable at all.

Decoy-Layer Connection Pooling — All Five Decoys

Decoy	Before (mean)	After (mean)	Δ
oracle-ai	6.15 ms	4.28 ms	**−30.4%**
vault-sql	0.69 ms	0.41 ms	**−41.4%**
ledger-trap	5.96 ms	4.20 ms	**−29.5%**
forge-ci	6.95 ms	6.71 ms	−3.4%
shadow-shell	7.91 ms	4.64 ms	**−41.3%**

Table IV, paper Section V.C

TAKEAWAY Real and consistent improvement on 4 of 5 decoys. `forge-ci` barely moves — and that is the correct result, because it always routes through the AI engine regardless of decoy-layer pooling.

Module 1, Runs 3–4 — Real-World Payload Datasets

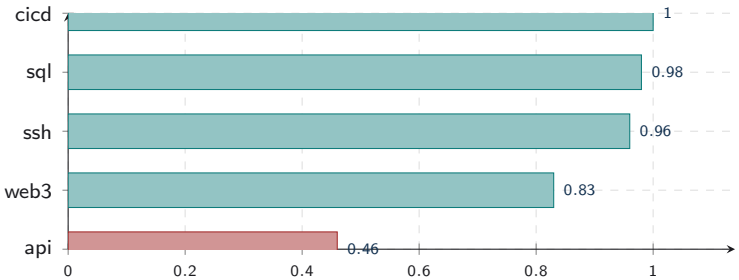
The hand-written dataset was replaced with **real, publicly-documented attack payloads** (payload-box, PayloadsAllTheThings, SecLists), scaled 100 → 200 → 1,328.

Metric	200-item set	1,328-item set
Dynamic latency (mean)	2.06 s	1.95 s
Concurrent throughput (10 sessions)	1.62 req/s	3.05 req/s
Response diversity — ssh	0.925	0.964
Response diversity — sql	1.00	0.978
Response diversity — web3	0.80	0.833
Response diversity — cicd	0.90	1.00
Response diversity — api	0.70	**0.46**
LLM-judge realism (mean)	4.03 / 5	**3.37 / 5**

Table VI, paper Section V.E–F

TAKEAWAY Latency and throughput *improved* at scale; realism *degraded*. The degradation is not spread evenly — it is concentrated in one service, which is the subject of the next three slides.

Response Diversity by Decoy Service



`api` is the clear outlier — on real XXE / Log4Shell strings the model collapses to a generic *"invalid request"* reply instead of emulating a plausible vulnerable-parser response.

Fig. 5, paper Section V.C, V.F

The Benchmark-Harness Bug

A genuine measurement bug, found and fixed during the real-dataset re-run.

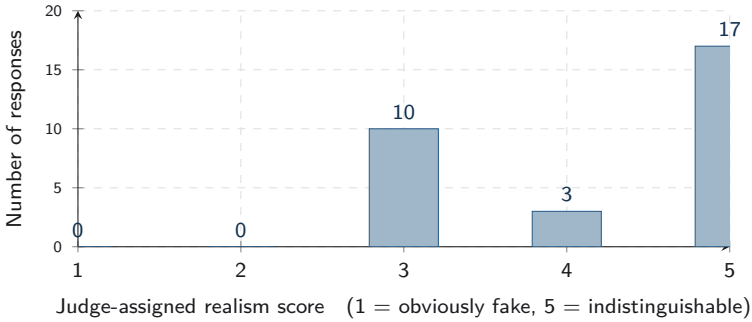
- The concurrent-throughput benchmark **reused payloads already sent once earlier in the same run**.
- Because the Redis cache key derives from *command + history*, these were guaranteed cache hits — so the "800 req/s" figure measured **Redis latency, not LLM latency**.
- **Fix:** append a fresh UUID nonce to every payload immediately before firing the concurrent batch.

Metric	First reported (buggy)	Corrected
Concurrent throughput (10 sessions)	800.3 req/s	3.05 req/s
Mean per-request latency	8.2 ms	2.62 s

Table V, paper Section V.E

TAKEAWAY Reported explicitly rather than silently substituted. A three-order-of-magnitude measurement error is itself a methodological finding — and a warning for anyone benchmarking a cached LLM pipeline.

Realism Score Distribution (Post-Optimisation, $n = 30$)



Bimodal, not centred: **17 responses are judged indistinguishable from a real system**, while 10 sit at a mediocre 3/5. **Zero** responses scored 1 or 2 at this dataset scale — a property that does *not* survive to the 1,328-item set.

Fig. 3, paper Section V.B

Key Finding — The API Decoy's Realism Failure

At the 1,328-item scale, **9 of 11** sampled `api` responses scored 1–2 / 5 — versus near-uniform 5/5 for `ssh`, `sql` and `web3` in the same sample.

Most consequential result

One sampled response was found to contain an **explicit AI safety disclaimer** ("Contains an explicit disclaimer about not generating harmful responses") — a *hard deception failure*, not a minor realism deduction. It signals "this is not a real system" to exactly the attentive attacker the framework is designed to hold.

- This failure was **invisible** in the 100-payload and even the 200-payload runs — it only became unambiguous at the largest scale. *Small benchmarks hide structural faults.*
- **Root cause:** the hosted model's own safety training bleeds through the deception persona on XML / JNDI-shaped adversarial input.
- **Actionable fix (specified, not yet applied):** XXE / Log4Shell / SSRF-aware few-shot examples plus an explicit anti-disclaimer instruction in `oracle-ai`'s system prompt.

Paper Section V.F, VI

Module 2 — Tamper-Evident Audit Trail, Measured

300 anchored events, 500 tampering trials, against a live Ganache chain — not a simulation.

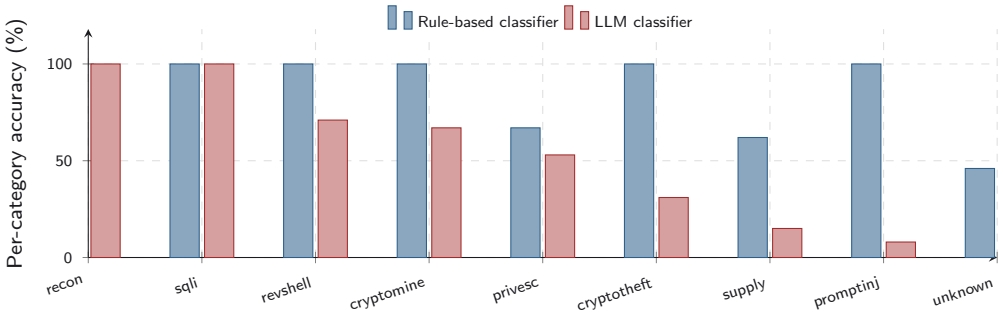
Measurement	Result	Reading
Anchor latency (mean / p95)	10.86 / 14.42 ms	on-chain commit dominates
SHA-256 hashing alone	0.007 ms	cryptography is *free* here
Verification (re-hash)	0.008 ms	checking is instant
Gas per anchor	21,510	a minimal-payload Ethereum tx
Anchoring success rate	300 / 300 = 100%	no dropped evidence
Tamper detection (500 trials)	**500 detected, 0 FP**	100% detection rate
Throughput seq. / conc. (10)	92.1 / 141.2 per s	concurrency gives 1.53×
On-chain retrievability	digest recovered	anyone can verify independently

Paper 3 (Ganache v7.9.2, chain ID 1337); measured 2026-08-18

TAKEAWAY Forensic integrity costs ≈11 ms per event and detects 100% of edits. Compared with a 1,350 ms LLM response, tamper-evidence is effectively free.

Module 3 — Can an LLM Replace the Rule-Based Classifier?

121 hand-labelled payloads, 9 intent categories, head-to-head on identical input.



Overall: rule-based **85.1%** vs. LLM **50.4%**; the two agree on only 56.2% of payloads. The LLM's default fallback label is "reconnaissance" — it mislabelled **all 13** benign/unknown payloads as active threats. Every verdict was anchored (121/121); 40/40 tamper trials detected.

Paper 4; Llama-3.1-8B, 1,669 ms mean LLM call + 24.8 ms anchor

Module 3 — Why This Negative Result Matters

- The obvious design instinct — "replace the brittle keyword rules with an LLM" — is **measurably wrong here**, by 35 percentage points, on the same inputs.
- The errors are **not evenly distributed**. They cluster in exactly the categories where a wrong answer is most expensive: prompt injection (8%), supply-chain (15%), crypto theft (31%), and benign traffic (0%).
- Over-classifying benign traffic as an active threat produces **alert fatigue**, which erodes operator trust more quietly than a missed detection does.
- This is why we anchor the *verdict*, not just the raw event: an AI artifact this fallible will sometimes be wrong, and **a wrong verdict that can be quietly edited afterwards is worse than no verdict at all** — it launders an error into something that looks authoritative.

TAKEAWAY Design principle we can defend with data: *an AI classifier in a security-decision role should be treated as a fallible witness, not as ground truth.*

Module 4 — Federated Threat Intelligence Without a Hub

Five concurrent honeypot nodes, 60 IOCs each, 300 anchored events on a live chain.

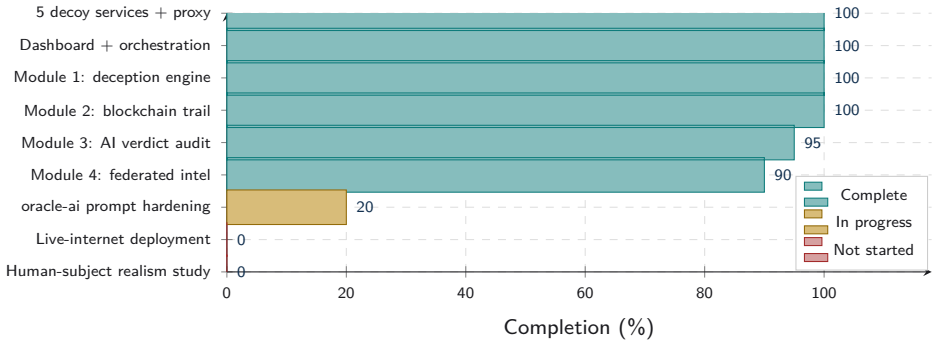
Idea: a campaign touches many decoys, but each honeypot sees only a fragment. The usual fix is a central threat-intel server — the one machine an adversary must reach to blind the whole network. **We remove it**: each node hashes its own indicators, tags the digest with its identity, and writes to a shared chain. No node ever calls another.

Measurement	Result	Reading
Anchor latency (mean / p95)	8.44 / 11.06 ms	per-node band 7.93–9.19 ms
Propagation to a blind observer	1.94 ms, 40/40	correlation without coordination
Cold-start reconstruction	all 220 IOCs	from chain history alone
Corroboration accuracy	195 single / 25 shared	no false merges or misses
Compromised-node forgery	**forgery caught**	original digest still on-chain
Throughput — 5 nodes vs. 1	228.6 vs. 98.6 per s	**2.32**× from concurrency

Paper 5; 35% ground-truth overlap from a 25-IOC campaign pool

TAKEAWAY A fresh node that never contacted any peer rebuilds the full campaign picture by reading the ledger — the single point of failure is genuinely gone, not merely replicated.

Project Status — Approximately 70% Complete



Done: full running stack, four measured evaluation modules, four paper drafts, all benchmarks reproducible against the live system. **Remaining:** prompt hardening + re-measurement, human-subject validation, longitudinal live deployment, learned intent classifier.

Discussion — What the Numbers Actually Mean

- **Not a drop-in replacement** for high-volume, low-interaction scanning honeypots — static-path systems remain appropriate there, and we say so.
- **Targeted use case:** high-interaction deception against a human operator or a sophisticated, low-volume tool, where multi-second latency is plausibly attributable to network or server load.
- **Concurrency is real but modest:** 10 concurrent sessions complete in 3.28 s wall-clock (not ≈19.5 s) — but far short of the originally (buggy) reported capacity.
- **The AI-persona failure is structural, not incidental:** any decoy built on a hosted, safety-tuned model inherits an alignment layer that is *directly adversarial* to the deception goal on concerning inputs.
- **Blockchain anchoring assumes the ledger is outside attacker reach** — reasonable for an internal Ganache deployment; not evaluated against an adversary who also compromises the chain client.
- **The cheap component won.** Across two independent experiments, the simple deterministic layer (static dictionary; keyword classifier) outperformed the sophisticated one on its own terms.

Paper Section VI

Limitations & Future Work

Limitations we acknowledge

- LLM-judge realism is an *automated proxy* — needs human-subject validation (protocol specified in paper Appendix A, not yet executed).
- Real-payload datasets (up to 1,328 items) are still *not live-traffic captures* — no multi-step sessions, timing, or evasion behaviour.
- `web3` and `cicd` datasets remain small (18 and 10 payloads) — no comparably large real corpus exists for those surfaces; a genuine coverage gap.
- Intent classification is keyword-based — will misclassify obfuscated or novel patterns.
- Federated results come from a *single local chain* with simulated nodes — not a geographically distributed deployment with real consensus latency and partitions.
- No longitudinal live-internet deployment; concurrency tested only at a single burst level.

Next steps, in priority order

1. Rewrite `oracle-ai`'s system prompt with XXE/JNDI-aware few-shot examples and an explicit anti-disclaimer instruction, then *re-measure* the realism score.
2. Replace the keyword classifier with a learned classifier and re-run the Module 3 head-to-head.
3. Execute the human-subject realism protocol; then a supervised live deployment.

Conclusion

1. We built and ran a **five-protocol, LLM-driven deception framework** behind one shared three-tier response engine — the first to apply a single LLM engine uniformly across heterogeneous protocols.
2. We made its forensic record **provable independently of the honeypot host**: 100% tamper detection over 500 trials at ≈11 ms per anchored event.
3. We extended that discipline to the **AI's own verdicts**, and then showed — with data — that those verdicts are wrong half the time, which is precisely why they must be anchored.
4. We removed the **central threat-intelligence hub**, showing that five independent nodes can corroborate a campaign by reading a shared ledger, with no node ever calling another.
5. Throughout, we **reported the failures**: a 3-order-of-magnitude benchmark bug, a realism regression bought with speed, and a hard deception failure caused by the model's own safety training.

TAKEAWAY The honest negative results are the contribution the field is missing — and they only became visible because we scaled the evaluation instead of stopping at a favourable number.

References I

1. A. Vetterl and R. Clayton, "Bitter Harvest: Systematically Fingerprinting Low- and Medium-interaction Honeypots at Internet Scale," *Proc. 12th USENIX WOOT*, 2018.
2. "Gotta Catch 'em All: A Multistage Framework for Honeypot Fingerprinting," *ACM Digital Threats: Research and Practice*, 2023, arXiv:2109.10652.
3. M. Sladić, V. Valeros, C. Catania and S. Garcia, "LLM in the Shell: Generative Honeypots," *Proc. IEEE EuroS&PW*, 2024, arXiv:2309.00155.
4. Z. Wang et al., "HoneyGPT: Breaking the Trilemma in Terminal Honeypots with Large Language Model," arXiv:2406.01882, 2024.
5. F. McKee and D. Noever, "Chatbots in a Honeypot World," arXiv:2301.03771, 2023.
6. "LLM Honeypot: Leveraging Large Language Models as Advanced Interactive Honeypot Systems," arXiv:2409.08234, 2024.
7. "Blockchain-Based Evidence Tracking System for Forensic Integrity and Secure Chain of Custody," IEEE, 2025.
8. "Blockchain-based tamper-proof and transparent investigation model for cloud VMs," *The Journal of Supercomputing*, Springer, 2022.

References II

9. "A novel intelligent cyber threat detection system with blockchain-enforced immutable forensic auditability," *PeerJ Computer Science*, 2025.
10. MITRE, "MITRE Engage: A Framework and Community for Cyber Deception," 2022.
11. Cowrie SSH/Telnet Honeypot, GitHub repository.
12. "SoK: Honeypots and Large Language Models — Systematization of a Growing Intersection," arXiv:2510.25939, 2025.
13. MISP — Open Source Threat Intelligence Platform and Open Standards for Threat Information Sharing.
14. OASIS, "STIX / TAXII: Structured Threat Information Expression and Trusted Automated Exchange of Intelligence Information," Version 2.1, 2021.

Thank You

Questions and discussion

Team 58 · Guide: Dr. Radhika G